ISET SFAX AU 2022/2023 SEMESTRE I		
DEVOIR SURVEILLE		
Matière : Programmation Objet		*Nb pages : 2*
Documents Non Autorisés	*Durée : 1 heure*	***Barème : 5 + 15***

Questions de cours (5 points)

1- Qu'est-ce que la **surcharge** de méthodes ? donner un exemple.

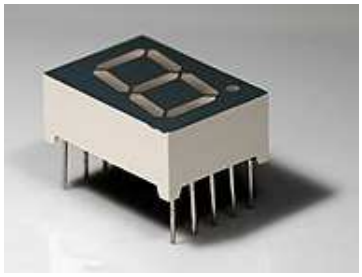
2- Qu'est-ce que le **bytecode** ? Qui **l'exécute** et **comment** il est exécuté ?

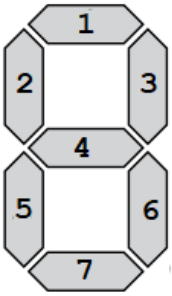
3- Qu'est-ce que l'**héritage** ? Quel est son **mot clé** utilisé en java ? donner un **exemple**.

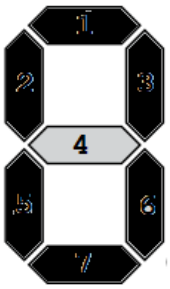
4- Quels sont les **attributs hérités** par la sous classe ?

Problème (15 points)

La classe "**SeptSegment**" permet de modéliser un afficheur sept segments pour afficher les entiers de 0 à 9. **SeptSegment** possède une marque ("ROHM Semiconductor", "kingbright Electronic",…), une couleur de ses segments ("Rouge", "Vert",…) et une valeur à afficher (de 0 à 9). **SeptSegment** possède aussi un attribut statique nbSeptSegment pour compter le nombre de SeptSegment instanciées. **SeptSegment** possède sept segments qui seront affichés ou non pour obtenir la valeur désirée. Par exemple pour afficher 0 l'état des segment de 1 à 7 est (true, true, true,false, true, true,true).







SeptSegment possède septs constantes qui sont : `SEG1 = " ------ \n", SEG2 = "|", SEG3 = " |\n", SEG4 = " ------ \n", SEG5 = "|", SEG6 = " |\n"` et `SEG7 = " ------ \n"`

Les méthodes de la classe SeptSegment sont décrites dans le tableau suivant :

Méthode	**Description**
`SeptSegment(…)`	Constructeur avec paramètres (marque et couleur)
`SeptSegment()`	Constructeur sans paramètres
`SeptSegment (…)`	Constructeur par copie

`getMarque()`	Accesseur à marque
`getCouleur()`	Accesseur à couleur
`getValeur()`	Accesseur à valeur
`getNbSeptSegment()`	Accesseur statique à nbSeptSegment
`setValeur (…)`	Modificateur de valeur
`reset()`	Permet de remettre valeur à zéro
`incrementer ()`	Permet d'incrémenter valeur d'une façon cyclique (0,1,2,…,8,9,0,1,…)
`afficherTout ()`	Affiche tous les attributs non statiques de la classe
`afficher()`	Permet d'afficher valeur en utilisant les sept segments. Cette méthode fait un « selon cas » sur la valeur et appelle afficher(…) qui prend en paramètre septs booléen (cette méthode est donnée) pour afficher la valeur correctement. Par exemple pour afficher 0 il faut passer en paramètre (true,true,true,false,true,true,true)
`afficher(…)`	Méthode privée qui prend en paramètre sept booléen pour affcher les segments désirées, son code est donné en bas

1- Donner le code de la classe SeptSegment (constantes, attributs et méthodes).

2- Donner une classe TestSeptSegment qui contient la méthode main et qui permet de :

   a. Instancier un **SeptSegment** "s1" avec le premier constructeur,

   b. Instancier un **SeptSegment** "s2" avec le deuxième constructeur,

   c. Instancier un **SeptSegment** "s3" avec le troisième constructeur,

   d. Afficher tous les attributs de "s1",

   e. Afficher (d'une façon statique) le nombre d'instances de **SeptSegment,**

   f. Remettre "s1" à zéro,

   g. Faire une boucle de langueur 40 qui afficher la valeur de "s1" et l'incrémente.

   h. Appeler afficher(…) qui prend en paramètre sept booléens pour afficher la valeur 8.

La méthode **privée** afficher(…) qui prend en paramètre sept booléen est :

```
private void afficher(boolean b1, boolean b2, boolean b3, boolean b4,
                      boolean b5, boolean b6, boolean b7) {
          if (b1)    System.out.print(SEG1);
          if (b2)    System.out.print(SEG2);
          if (b3)    System.out.print(SEG3);
          if (b4)    System.out.print(SEG4);
          if (b5)    System.out.print(SEG5);
          if (b6)    System.out.print(SEG6);
          if (b7)    System.out.print(SEG7);
}
```

